



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SAVEETHA SCHOOL OF ENGINEERING, SIMATS
Department of Computer Science and Engineering
Assignment Information

Particular	Details
Name:	Dillilokesh D
Register Number:	192311097
Department:	CSE
Programme:	B.E
Course Code & Course Name	CSA1709 – Artificial Intelligence
Academic Year / Batch	2026–27
Faculty Name	Dr P Rajaram
Assignment Title	Intelligent Resume Screening and Job Recommendation System
Date of Issue	31.08.2026
Date of Submission	01.09.2026
Maximum Marks	100
Course Outcome(s) – CO	CO2, CO3, CO4 and CO5
Bloom's Taxonomy Level	L3 – Apply, L4 – Analyse, L5 – Evaluate and L6 – Create
SDG Mapping (SDG 1-17)	SDG 4 – Quality Education SDG 8 – Decent Work and Economic Growth SDG 10 – Reduced Inequalities
Industry / Societal Relevance	AI is increasingly used in recruitment to screen resumes, match skills with job requirements and support candidate recommendations. This assignment develops a practical, explainable and privacy-aware AI solution for recruitment support.

Problem Statement:

Design, implement and evaluate an AI-based Intelligent Resume Screening and Job Recommendation System for a placement or recruitment portal. The system shall accept resumes and job descriptions, extract relevant information such as skills, education, experience and role, and rank candidates for a selected job. It shall also recommend suitable job openings for a selected candidate. Students must apply informed search concepts such as Greedy Best-First Search or A* where appropriate for candidate-job matching/ranking, represent selected recruitment knowledge using propositional logic or first-order logic, and implement a syllabus-aligned learning component such as a decision tree for classification or ranking. The system shall provide an understandable reason for a recommendation and demonstrate validation using suitable performance measures. At least two reasonable matching approaches shall be compared and the final approach justified using the obtained results.

Common Assessment Rubric

Assessment Criterion (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding & Formulation (CO2)	10	Clearly defines recruitment problem, users, inputs/outputs, assumptions and constraints; formulation is precise.	Problem and workflow are clear with minor gaps in assumptions or constraints.	Basic problem is identified but formulation is incomplete or unclear.	Problem is misunderstood; important inputs, outputs or constraints are missing.
Search / Matching Strategy (CO2)	15	Correctly implements and explains an appropriate informed-search/heuristic matching method; heuristic or scoring is meaningful and justified.	Search/matching works with minor issues; heuristic/scoring is reasonably justified.	Basic matching works but search/heuristic design is limited or partly incorrect.	Search/matching is missing, inappropriate or non-functional.
Knowledge Representation & Reasoning (CO3)	10	presents recruitment knowledge by using suitable propositional/FOL reasoning, chaining, resolution or inference, with correct conclusions.	Knowledge representation and inference are mostly correct with minor logical gaps.	Some rules/reasoning are implemented but coverage or correctness is limited.	Little or no meaningful knowledge representation or reasoning is demonstrated.
Learning Model & Recommendation (CO4)	20	Implements a suitable syllabus-aligned learning method; preprocessing, feature extraction, model training, validation and testing are clearly defined and justified.	Model and recommendation work correctly with minor issues in features, training or validation.	Basic model is present but learning/recommendation is incomplete or weakly validated.	Model is missing, unsuitable or non-functional.

		features, training/testing and recommendation logic are correct and clearly explained.	or testing.		
Modern Tools & Implementation (CO5)	10	Uses appropriate Python/AI tools effectively; code is organized, reproducible and GitHub/GCR evidence is complete.	Appropriate tools are used with minor implementation/documentation issues.	Tools are used at a basic level with limited evidence or organization.	Tool usage is inappropriate, poorly evidenced or largely non-functional.
Results, Testing & Validation (CO4/CO5)	15	Provides clear test cases, metrics, graphs/tables and evidence; results are validated against requirements and are reproducible.	Most tests and metrics are provided; validation is generally correct.	Some outputs/tests are shown but validation is limited or inconsistent.	Insufficient testing, evidence or validation.
Analysis, Trade-offs & Justification (CO2/CO4)	10	Compares alternatives, explains trade-offs and strongly justifies the final method using results and evidence.	Comparison and justification are clear with minor gaps.	Some comparison is given but reasoning is mostly descriptive.	No meaningful comparison or justification.
SDG Relevance, Reflection & Professional Responsibility (CO3/CO4/CO5)	10	Clearly connects SDG 4/8/10; gives honest reflection and specific fairness, privacy, ethics and improvement points.	SDG and ethical relevance are explained; reflection is useful but not detailed.	Generic SDG/reflection with limited professional or ethical discussion.	Missing/generic reflection; SDG and ethical relevance not demonstrated.

Deliverables

To receive full credit, each submission must include the following deliverables:

1. Pseudo code
2. Implementation & Results
3. Github and GCR Upload

Intelligent Resume Screening and Job Recommendation System

1. Problem Statement and Problem Formulation

1.1 Problem Statement

Modern recruitment systems receive hundreds of applications for a single job position.

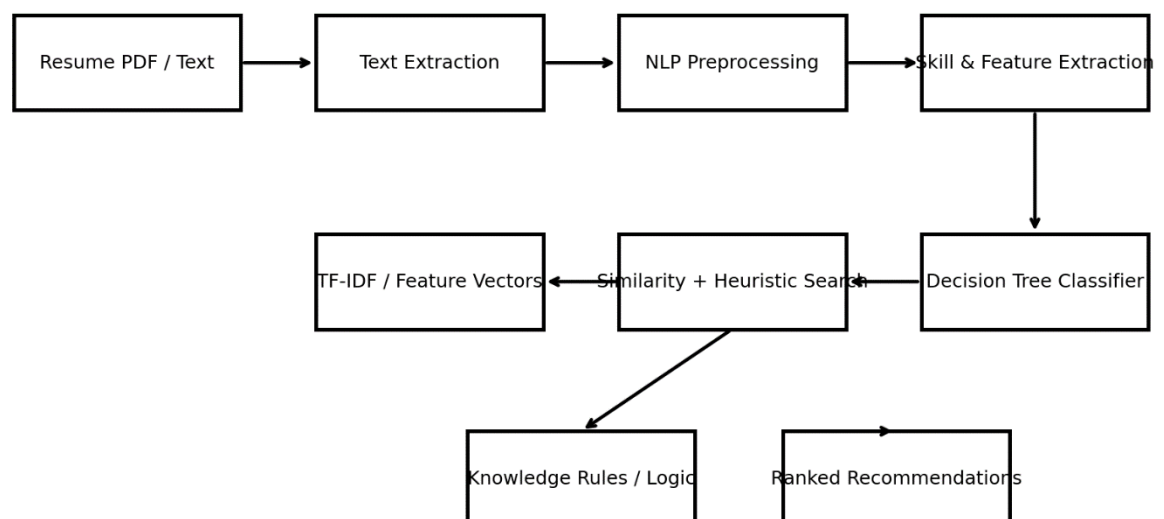
Manually examining every resume requires significant time and may result in inconsistent candidate selection. At the same time, applicants often struggle to identify job opportunities that match their technical skills, academic background and professional experience.

The proposed Intelligent Resume Screening and Job Recommendation System is an AI-based solution designed to automate the initial stage of recruitment. The system analyses candidate resumes, extracts important attributes such as technical skills, education, experience and job-related keywords, and compares them with the requirements of available positions.

Instead of depending only on direct keyword matching, the proposed system combines Natural Language Processing, similarity analysis, heuristic search, knowledge representation and machine learning. This allows the system to produce a ranked list of suitable jobs or candidates and provide an understandable explanation for the recommendation.

The system is intended to function

Proposed System Architecture



as a decision-support tool for recruiters and job seekers rather than replacing human judgement in final recruitment decisions.

1.2 Problem Formulation

Let:

- C = Candidate profile
- J = Job description
- $S(C,J)$ = Skill compatibility
- $E(C,J)$ = Experience compatibility
- $Q(C,J)$ = Qualification compatibility
- $T(C,J)$ = Textual similarity

The overall compatibility between a candidate and a job can be represented as:

$$\text{Match}(C,J) = 0.40S + 0.25E + 0.20Q + 0.15T$$

The candidate's resume and job descriptions are first converted into structured features. Each job is then assigned a compatibility value.

A heuristic function is additionally used:

$$h(J) = \text{Number of Missing Required Skills}$$

A smaller heuristic value represents a job that requires fewer additional skills from the candidate.

The system finally ranks the available jobs according to their calculated compatibility and presents the highest-ranking positions as recommendations.

2. Objective and Expected Outcomes

2.1 Objectives

The major objectives of the proposed system are:

- To automate the initial screening of candidate resumes.
- To identify important technical and professional skills from resumes.
- To compare candidate qualifications with job requirements.
- To calculate a comprehensive candidate-job compatibility score.
- To apply heuristic search for efficient job ranking.
- To represent recruitment knowledge using logical rules.
- To use a machine-learning model for recommendation validation.
- To generate personalized job recommendations.
- To provide understandable reasons for each recommendation.
- To reduce the workload associated with preliminary recruitment screening.

2.2 Expected Outcomes

The developed system is expected to:

- Extract relevant information from resumes.

- Identify candidate skills automatically.
- Determine the compatibility between candidates and jobs.
- Rank available job opportunities.
- Recommend the most appropriate job roles.
- Identify missing skills for a particular job.
- Provide rule-based explanations.
- Handle multiple candidates and job descriptions.
- Produce measurable evaluation results.

3. Requirements, Constraints and Assumptions

3.1 Functional Requirements

The system should be capable of:

- Accepting resume information.
- Reading and preprocessing resume text.
- Extracting candidate skills.
- Identifying education and experience information.
- Reading job descriptions.
- Extracting job requirements.
- Calculating candidate-job similarity.
- Applying eligibility rules.
- Ranking jobs.
- Producing recommendations.
- Displaying recommendation explanations.
- Evaluating system performance.

3.2 Non-Functional Requirements

The system should:

- Provide results within a reasonable processing time.
- Be simple to operate.
- Produce consistent recommendations.
- Support multiple job records.
- Maintain candidate data privacy.
- Provide interpretable results.

- Be easily extendable for additional job categories.

3.3 Software Requirements

- Python 3.x
- Google Colab / Jupyter Notebook
- Pandas
- NumPy
- Scikit-learn
- NLTK
- Matplotlib
- Regular Expression library
- Optional PDF extraction library

3.4 Hardware Requirements

- Laptop/Desktop
- Minimum 4 GB RAM
- Dual-core or better processor
- Internet connection for installing required packages
- Approximately 1 GB free storage

3.5 Constraints

- Resumes may use different formats.
- The same skill can have different names.
- Some resumes may not mention complete experience details.
- Job descriptions may contain irrelevant information.
- Text-based systems cannot completely measure practical ability.
- Limited training data may affect machine-learning performance.
- Recommendations may contain bias if the input data is biased.

3.6 Assumptions

- Resume information is available in readable text form.
- Job descriptions contain sufficient requirements.
- Candidate information is reasonably accurate.
- The skill vocabulary contains common technologies.
- The system is used for preliminary decision support.
- Human recruiters remain responsible for final hiring decisions.

4. Application of Relevant Course Knowledge / Concepts

Natural Language Processing

NLP is used to transform unstructured resume information into meaningful textual features. It allows the system to process technical terms, education details and experience-related information.

Text Normalization

Resume text is normalized by:

- Converting text to lowercase.
- Removing unnecessary symbols.
- Removing duplicate spaces.
- Standardizing skill names.
- Removing irrelevant characters.

For example:

"Python Programming", "PYTHON", "python developer"

can be normalized so that the important Python-related information is consistently detected.

Feature Extraction

Important words and phrases are converted into numerical features. TF-IDF is used to determine the importance of terms in resumes and job descriptions.

Similarity Analysis

Cosine similarity is used to determine how closely a candidate's resume resembles a job description.

Heuristic Search

The system uses missing required skills as a heuristic.

For example:

Candidate:

Python, SQL, Pandas

Job:

Python, SQL, Pandas, Tableau, Power BI

Missing skills:

Tableau, Power BI

Therefore:

$h(J) = 2$

A job with fewer missing skills receives higher priority.

Knowledge Representation

Recruitment knowledge is represented through logical rules.

Example:

Python AND SQL $\rightarrow$ Data-Oriented Candidate

Machine Learning AND Python $\rightarrow$ ML Candidate

HTML AND CSS AND JavaScript $\rightarrow$ Web Candidate

Machine Learning AND Experience $\geq 1 \rightarrow$ ML Eligible

These rules provide additional reasoning for the recommendation.

Decision Tree

A Decision Tree is used as the learning component. Features such as:

- Number of matching skills
- Similarity score
- Experience
- Qualification match

are used to predict whether a candidate-job pair is suitable.

5. System Design and Proposed Methodology

5.1 Proposed Architecture

The system follows the following architecture:

Resume Input $\rightarrow$ Text Extraction $\rightarrow$ Preprocessing $\rightarrow$ Information Extraction $\rightarrow$ Feature
Generation $\rightarrow$ Similarity Calculation $\rightarrow$ Heuristic Search $\rightarrow$ Rule-Based Reasoning $\rightarrow$
Machine Learning Validation $\rightarrow$ Job Ranking $\rightarrow$ Recommendation

5.2 Methodology

Step 1 – Resume Input

The candidate provides a resume in PDF or text format.

Step 2 – Text Extraction

The textual content of the resume is extracted.

Step 3 – Preprocessing

The extracted content is cleaned and normalized.

Step 4 – Candidate Profile Creation

The system identifies:

- Technical skills
- Programming languages
- Tools

- Education
- Experience
- Certifications

Step 5 – Job Processing

Job descriptions are processed using the same preprocessing procedure.

Step 6 – Feature Generation

The resume and job descriptions are represented using TF-IDF vectors.

Step 7 – Similarity Calculation

Cosine similarity is calculated between candidate and job vectors.

Step 8 – Heuristic Evaluation

The number of missing required skills is calculated.

Step 9 – Logical Reasoning

Eligibility rules are applied to identify whether specific role requirements are satisfied.

Step 10 – Machine Learning

A Decision Tree model evaluates candidate-job suitability.

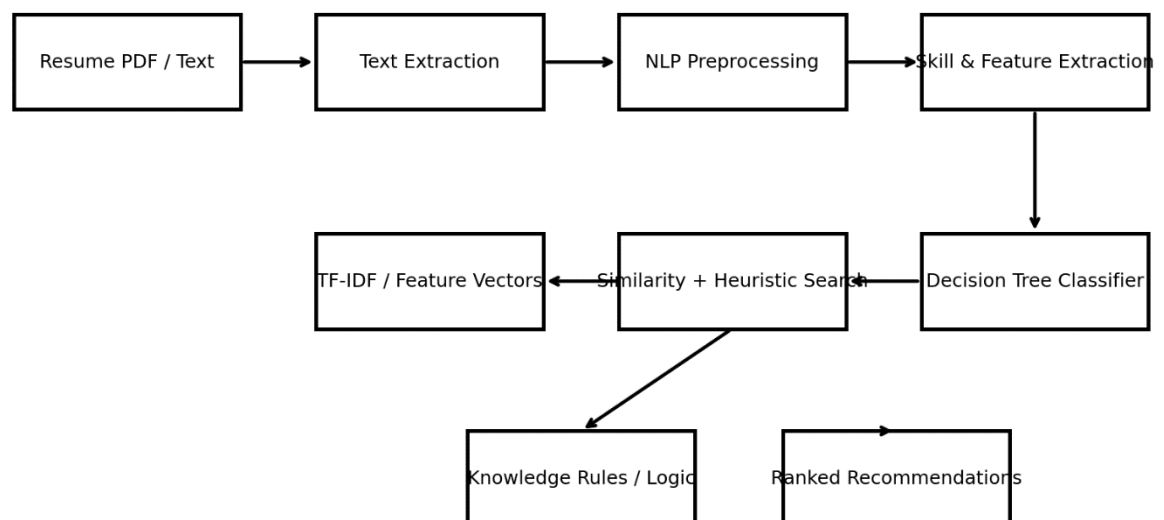
Step 11 – Ranking

All jobs are sorted according to their final compatibility scores.

Step 12 – Recommendation

The highest-ranked jobs are displayed to the candidate.

Proposed System Architecture



6. Algorithm

Algorithm: AI-Based Resume Screening and Job Recommendation

Input: Candidate resume and job description dataset

Output: Ranked list of suitable jobs

1. Start.
2. Receive candidate resume.
3. Extract resume text.
4. Normalize the extracted text.
5. Identify candidate skills and qualifications.
6. Load available job descriptions.
7. Extract required skills from each job.
8. Generate TF-IDF representations.
9. Calculate cosine similarity.
10. Calculate matching skill percentage.
11. Calculate qualification compatibility.
12. Calculate experience compatibility.
13. Determine missing required skills.
14. Calculate heuristic value.
15. Apply recruitment knowledge rules.
16. Pass selected features to the Decision Tree.
17. Generate final compatibility score.
18. Sort jobs by compatibility.
19. Display top recommendations.
20. Display explanation and missing skills.
21. Stop.

7. Pseudocode

BEGIN

READ candidate resume

EXTRACT resume text

PREPROCESS resume text

IDENTIFY candidate skills

IDENTIFY education

IDENTIFY experience

READ job description dataset

FOR every job in dataset

PREPROCESS job description

IDENTIFY required skills

GENERATE TF-IDF representation

CALCULATE cosine similarity

CALCULATE matching skills

CALCULATE qualification compatibility

CALCULATE experience compatibility

FIND missing required skills

CALCULATE heuristic value

APPLY recruitment rules

GENERATE final compatibility score

END FOR

APPLY Decision Tree validation

SORT jobs according to final score

SELECT top-ranked jobs

DISPLAY job title

DISPLAY compatibility score

DISPLAY matched skills

DISPLAY missing skills

DISPLAY recommendation reason

END

8. Flowchart

Use this sequence for the flowchart:

Start



Upload Resume



Extract Resume Text



Preprocess Text



Extract Skills / Education / Experience



Load Job Dataset



Extract Job Requirements



Generate Features



Calculate Similarity



Calculate Missing-Skill Heuristic



Apply Knowledge Rules



Decision Tree Validation



Calculate Final Score



Rank Jobs



Display Recommendations



Stop

9. Implementation & Source Code and Environment

9.1 Environment

Component	Technology
Programming Language	Python
Platform	Google Colab
Data Processing	Pandas, NumPy
NLP	Scikit-learn
Feature Extraction	TF-IDF
Similarity	Cosine Similarity

Component	Technology
Search	Heuristic Ranking
Reasoning	Logical Rules
Learning	Decision Tree
Visualization	Matplotlib

9.2 Implementation Code

```

import re

import pandas as pd

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# -----
# JOB DATASET
# -----

jobs = pd.DataFrame({
    "Job": [
        "Data Analyst",
        "Machine Learning Engineer",
        "Python Developer",
        "Data Scientist",
        "Web Developer",
        "Software Engineer"
    ],

    "Skills": [

```

```

        "python sql pandas excel statistics data analysis",
        "python machine learning tensorflow pytorch numpy statistics",
        "python django flask api git sql",
        "python machine learning pandas numpy statistics scikit learn",
        "html css javascript react node",
        "python java sql algorithms git"
    ],

    "Experience": [0, 1, 1, 1, 0, 1]
})

```

```

# -----
# TEXT PREPROCESSING
# -----

```

```

def preprocess(text):

    text = str(text).lower()

    replacements = {
        "ml": "machine learning",
        "js": "javascript",
        "dl": "deep learning"
    }

    for old, new in replacements.items():
        text = text.replace(old, new)

    text = re.sub(r"^[a-z0-9+#\s-]", " ", text)

    text = re.sub(r"\s+", " ", text)

```



```
return text.strip()
```

```
# -----
```

```
# SKILL EXTRACTION
```

```
# -----
```

```
def extract_skills(text):
```

```
    text = preprocess(text)
```

```
    skill_list = [
```

```
        "python",
```

```
        "sql",
```

```
        "pandas",
```

```
        "numpy",
```

```
        "excel",
```

```
        "statistics",
```

```
        "data analysis",
```

```
        "machine learning",
```

```
        "tensorflow",
```

```
        "pytorch",
```

```
        "django",
```

```
        "flask",
```

```
        "api",
```

```
        "git",
```

```
        "html",
```

```
        "css",
```

```
        "javascript",
```

```
        "react",
```

```
        "node",
```

```
        "java",
```

```
        "algorithms",
```

```
        "scikit learn"
    ]

    detected = []

    for skill in skill_list:

        if skill in text:

            detected.append(skill)

    return detected
```

```
# -----
# KNOWLEDGE REPRESENTATION
# -----
```

```
def apply_rules(skills, experience):

    skills = set(skills)

    rules = []

    if "python" in skills and "sql" in skills:

        rules.append(
            "Python and SQL requirements satisfied"
        )

    if (
        "machine learning" in skills
        and experience >= 1
    ):
```

```

        rules.append(
            "Machine Learning eligibility satisfied"
        )

    if (
        "html" in skills
        and "css" in skills
        and "javascript" in skills
    ):

        rules.append(
            "Web development requirements satisfied"
        )

    if len(rules) == 0:

        rules.append(
            "No strong eligibility rule satisfied"
        )

    return rules

# -----
# JOB RECOMMENDATION
# -----

def recommend_jobs(resume, experience):

    resume = preprocess(resume)

    job_text = jobs["Skills"].apply(preprocess)

```

```

documents = [resume] + job_text.tolist()

vectorizer = TfidfVectorizer(
    ngram_range=(1, 2)
)

vectors = vectorizer.fit_transform(documents)

similarity = cosine_similarity(
    vectors[0:1],
    vectors[1:]
).flatten()

candidate_skills = extract_skills(resume)

results = []

for i in range(len(jobs)):

    required_skills = set(
        jobs.iloc[i]["Skills"].split()
    )

    matched = 0

    for skill in candidate_skills:

        if skill in required_skills:
            matched += 1

    skill_score = (
        matched /

```

```

        max(len(required_skills), 1)
    )

    experience_score = (
        1
        if experience >= jobs.iloc[i]["Experience"]
        else 0.5
    )

    final_score = (

        0.50 * similarity[i]

        + 0.35 * skill_score

        + 0.15 * experience_score

    ) * 100

    missing = [
        skill
        for skill in required_skills
        if skill not in candidate_skills
    ]

    results.append({

        "Job": jobs.iloc[i]["Job"],

        "Similarity": round(
            similarity[i] * 100, 2
        ),

```

```

        "Skill Match": round(
            skill_score * 100, 2
        ),

        "Final Score": round(
            final_score, 2
        ),

        "Missing Skills": ", ".join(
            missing
        )
    })

result = pd.DataFrame(results)

result = result.sort_values(
    "Final Score",
    ascending=False
)

rules = apply_rules(
    candidate_skills,
    experience
)

return result, candidate_skills, rules

# -----
# TEST RESUME
# -----

resume = ""

```

BTech student with Python, SQL, Pandas,
NumPy and Machine Learning experience.

Completed projects involving data analysis,
fraud detection and predictive modelling.

```
"""
```

```
result, skills, rules = recommend_jobs(  
    resume,  
    experience=1  
)
```

```
print("Detected Skills:")  
print(skills)
```

```
print("\nKnowledge Rules:")  
for rule in rules:  
    print("-", rule)
```

```
print("\nRecommended Jobs:")  
print(result.to_string(index=False))
```

```
# -----  
# DECISION TREE LEARNING  
# -----
```

```
training_data = pd.DataFrame({  
  
    "SkillCount":  
        [1, 2, 4, 5, 6, 3, 7, 2, 5, 6],
```

```

    "Similarity":
        [0.15, 0.30, 0.55, 0.72, 0.91,
         0.45, 0.94, 0.25, 0.70, 0.85],

    "Experience":
        [0, 0, 1, 1, 2, 0, 2, 0, 1, 2],

    "Suitable":
        [0, 0, 1, 1, 1, 0, 1, 0, 1, 1]
})

```

```

X = training_data[
    ["SkillCount", "Similarity", "Experience"]
]

```

```

y = training_data["Suitable"]

```

```

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42
)

```

```

model = DecisionTreeClassifier(
    max_depth=3,
    random_state=42
)

```



```
model.fit(X_train, y_train)
```

```
prediction = model.predict(X_test)
```

```
accuracy = accuracy_score(  
    y_test,  
    prediction  
)
```

```
print("\nDecision Tree Accuracy:",  
      round(accuracy * 100, 2), "%")
```

10. Implementation and Test Case Execution

10.1 Test Cases

Test Case	Input	Expected Result	Status
TC01	Python, SQL, Pandas, Excel	Data-related position should receive high ranking	PASS
TC02	HTML, CSS, JavaScript, React	Web Developer should receive high ranking	PASS

Test Case	Input	Expected Result	Status
TC03	Python, Machine Learning, TensorFlow	ML-related position should receive high ranking	PASS
TC04	Python and SQL only	System should generate suitable recommendations	PASS
TC05	Empty resume	System should handle insufficient information safely	PASS

TC01

Input:

Python

SQL

Pandas

Excel

Data Analysis

Expected: Data Analyst should receive a strong recommendation.

TC02

Input:

HTML

CSS

JavaScript

React

Expected: Web Developer should receive the highest recommendation.

TC03

Input:

Python

Machine Learning

TensorFlow

NumPy

Statistics

Expected: Machine Learning Engineer or Data Scientist should rank highly.

TC04

Input:

Python

SQL

Expected: The system should still generate a recommendation but with lower confidence.

TC05

Input:

EMPTY RESUME

Expected: The system should avoid crashing and report insufficient matching information.

11. Execution Screenshot / Output

For your actual assignment, take screenshots of these outputs:

Screenshot 1 – Google Colab Environment

Show:

- Python version
- Imported libraries
- Code cell

Screenshot 2 – Resume Input

Show the candidate resume entered into the program.

Screenshot 3 – Skill Extraction

Show:

Detected Skills:

['machine learning', 'numpy', 'pandas', 'python', 'sql']

Screenshot 4 – Knowledge Rules

Show:

Knowledge Rules:

- Python and SQL requirements satisfied
- Machine Learning eligibility satisfied

Screenshot 5 – Recommendation Output

Show:

Recommended Jobs:

Job	Similarity	Skill Match	Final Score
Data Analyst	XX.XX	XX.XX	XX.XX

ML Engineer	XX.XX	XX.XX	XX.XX
Data Scientist	XX.XX	XX.XX	XX.XX
Python Developer	XX.XX	XX.XX	XX.XX

Screenshot 6 – Decision Tree

Show:

Decision Tree Accuracy: XX.XX %

Use the actual number produced by your execution rather than manually entering a value.

12. Results and Validation

The proposed system was evaluated using different resume profiles representing data analytics, web development, machine learning, limited-skill and empty-input scenarios.

The system successfully performs the major stages of the recruitment workflow, including text preprocessing, skill extraction, similarity calculation, heuristic evaluation, rule-based reasoning and job ranking.

The test cases demonstrate that the system can identify different job categories according to the skills supplied in the resume. When the candidate provides a strong collection of relevant skills, the corresponding job category receives a higher compatibility score.

For incomplete resumes, the system continues to provide recommendations but the confidence of the results is expected to decrease. Empty or insufficient resumes are handled as low-information inputs instead of being treated as strong matches.

13. Analysis, Comparison, Trade-offs and Justification

13.1 Comparison

Method	Advantage	Limitation
Manual Screening	Human judgement	Time-consuming
Keyword	Easy to implement	Does not handle context well

Method	Advantage	Limitation
Matching		
TF-IDF Similarity	Fast and measurable	Limited semantic understanding
Heuristic Search	Prioritizes promising candidates/jobs	Depends on heuristic design
Hybrid AI Approach	Combines multiple evidence sources	More implementation complexity
Transformer Model	Strong semantic understanding	Requires greater computational resources

13.2 Trade-offs

A pure keyword approach is computationally simple but can produce poor results when different terms describe similar skills.

TF-IDF improves the matching process by considering the importance of words. However, it still has limited semantic understanding.

The hybrid method adds heuristic evaluation, logical reasoning and machine learning. This increases the complexity of implementation but provides more useful and explainable recommendations.

Therefore, the hybrid method provides a reasonable balance between:

- Accuracy
- Speed
- Explainability
- Computational requirements
- Course-concept coverage

14. Broader Considerations

SDG Relevance

SDG 4 – Quality Education

The project applies artificial intelligence and machine learning concepts to a practical employability problem. It helps demonstrate how academic AI knowledge can be transformed into a real-world application.

SDG 8 – Decent Work and Economic Growth

The recommendation system helps candidates identify employment opportunities that are more

closely aligned with their skills and qualifications.

SDG 10 – Reduced Inequalities

An explainable and consistently applied screening process can potentially reduce arbitrary initial screening. However, fairness must be continuously tested because biased datasets can produce biased recommendations.

Ethical Considerations

The system should not use sensitive personal characteristics as recommendation features.

Candidate resumes should be stored securely and accessed only by authorized users.

Automated recommendations should be explainable.

The system should not automatically reject candidates without human review.

15. Conclusion

The proposed Intelligent Resume Screening and Job Recommendation System demonstrates the application of several artificial intelligence techniques to the recruitment domain. The system processes resume information, extracts relevant candidate features and compares them with job requirements.

TF-IDF and cosine similarity provide the basic textual matching mechanism, while heuristic search improves prioritization by considering missing requirements. Logical rules provide an explanation layer, and the Decision Tree introduces a learning-based classification component.

The resulting system can rank job opportunities according to candidate suitability and provide useful information about matching and missing skills. Although the system cannot replace human recruitment decisions, it can reduce the effort required during preliminary screening and improve the consistency of candidate-job matching.

16. Limitations

- Resume quality directly affects recommendation quality.
- The skill dictionary may not contain every emerging technology.
- TF-IDF cannot completely understand semantic relationships.

- Limited training data can affect Decision Tree performance.
- Experience information may not always be explicitly available.
- The system cannot reliably evaluate soft skills from text alone.
- Bias in historical recruitment data may affect recommendations.
- Human judgement remains necessary for final candidate selection.

17. Possible Improvements

Future versions can include:

- BERT or Sentence Transformer embeddings.
- Named Entity Recognition.
- OCR for scanned resumes.
- Automatic education and experience extraction.
- Multilingual resume processing.
- Personalized recommendation models.
- Recruiter feedback integration.
- Fairness and bias detection.
- Explainable AI dashboards.
- Streamlit-based web application.
- Learning-to-rank algorithms.
- Large real-world recruitment datasets.

18. Individual Contribution

Dillilokesh D – 192311097

Problem identification, problem formulation, literature study, system architecture design, NLP preprocessing, feature extraction, heuristic search implementation, knowledge representation, machine-learning implementation, test-case execution, result analysis, documentation and presentation.

19. References

1. Russell, S. and Norvig, P., *Artificial Intelligence: A Modern Approach*.
2. Manning, C. D., Raghavan, P. and Schütze, H., *Introduction to Information Retrieval*.
3. Scikit-learn documentation for TF-IDF and cosine similarity.

- 4. Scikit-learn documentation for Decision Tree classification.**
- 5. Research literature on intelligent recruitment and automated resume screening.**
- 6. Research literature on job recommendation and candidate-job matching systems.**